

Attention, transformers, and generative models

Three lectures of mechanics, in one session

Fall 2026 · Week 11 · hold this after Lecture 18; Assignment 5 is due Tue 11/24

What this session is for

This is the longest handout of the term and the last one about content, because the calendar leaves no alternative: Assignment 5 covers Lectures 15–18, it is due on Tuesday 11/24, Thanksgiving recess begins the day after, and the final exam is the first day back. So Lectures 16, 17 and 18 have to share one session.

Read it in three passes rather than one. Section 1 (attention) is the part everything else depends on, and it is worth doing slowly with a pencil. Section 2 (generation) is short. Section 3 (VAEs and diffusion) is two objectives, each of which is five lines of code once you have seen it written down.

The one idea, and it covers all three lectures. Every model here is built from the same two moves: compute a set of *weights* that say how much each thing should contribute, and take a weighted average. Attention does it over positions in a sequence. A VAE does it over a distribution of latent codes. Diffusion does it over time steps of a noising process. The differences are in what is being averaged and what determines the weights.

1 Attention

1.1 Tokens are a matrix

A sequence of T tokens, each embedded as a d -dimensional vector, is a matrix $\mathbf{X}$ of shape (T, d) . Everything in a transformer operates on that matrix. Note what has been given up compared to an RNN: there is no order in a matrix's rows unless you put it there, which is why positional encodings exist.

1.2 Q, K, V are three views of the same input

$\mathbf{Q} = \mathbf{X} @ \mathbf{W}_q \# (T, d_k)$ *what each position is looking for*
 $\mathbf{K} = \mathbf{X} @ \mathbf{W}_k \# (T, d_k)$ *what each position offers*
 $\mathbf{V} = \mathbf{X} @ \mathbf{W}_v \# (T, d_v)$ *what each position would contribute*

All three are linear projections of the *same* $\mathbf{X}$ — that is what makes it *self*-attention. The names are a retrieval metaphor: a query is matched against keys, and the matching keys' values are returned.

1.3 Scaled dot-product attention, in four lines

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}$$

```
scores = Q @ K.transpose(-2, -1) / math.sqrt(d_k) # (T, T)
if mask is not None:
    scores = scores.masked_fill(mask == 0, float('-inf'))
A = torch.softmax(scores, dim=-1) # (T, T), rows sum to 1
out = A @ V # (T, d_v)
```

Track the shapes, because they are the whole thing:

Step	Shape	Meaning
$\mathbf{Q}\mathbf{K}^\top$	(T, T)	every position against every position
after softmax	(T, T)	each row is a distribution over positions
$\mathbf{A}\mathbf{V}$	(T, d_v)	each position: a weighted average of all values

Three details that are not decoration. The $\sqrt{d_k}$: dot products of d_k -dimensional vectors grow like $\sqrt{d_k}$, and a softmax of large numbers saturates to a one-hot vector with almost no gradient. Remove it and training fails for large d_k . **dim=-1**: the softmax runs across keys, so each row is a distribution over what to attend to. Softmax over the wrong axis produces nonsense that runs perfectly. **The mask is $-\infty$, not 0**: it must be applied *before* the softmax, because zeroing afterwards leaves rows that no longer sum to one.

1.4 Multi-head, which is only a reshape

Split the d dimensions into H groups, run the same operation on each, and concatenate the results. It lets different heads attend to different things.

```
# (N, T, d) -> (N, H, T, d//H)
Q = Q.view(N, T, H, d // H).transpose(1, 2)
# ... attention exactly as above, batched over the H axis ...
out = out.transpose(1, 2).reshape(N, T, d) # concatenate the heads back
```

The `transpose(1, 2)` before and after is the only subtle part. It puts the head axis next to the batch axis so that the matrix multiplies treat heads as independent problems.

1.5 Reading an attention map, carefully

The (T, T) matrix $\mathbf{A}$ is tempting to plot and interpret. Three cautions, all of which have caused published overclaims:

- There are H heads per layer and many layers, so “the attention map” is a choice among dozens, and they disagree.

- Attention weights are *not* feature importances. A position can be attended to heavily and contribute little, because what it contributes is its *value* vector, not itself.
- The residual stream carries information around the attention block entirely, so a position with zero attention is not disconnected.

The one-sentence version. An attention map tells you where information was *routed*, not what the model *used*. Recitation 9's whole discipline applies here too.

2 Autoregressive generation

Training a language model is next-token prediction with teacher forcing – exactly the mechanism from Recitation 10, at scale. Generation is a loop:

```
for _ in range(n_new):
    logits = model(idx)[: , -1, :] / temperature # last position only
    if top_k is not None:
        v, _ = torch.topk(logits, top_k)
        logits[logits < v[:, [-1]]] = float('-inf')
    probs = torch.softmax(logits, dim=-1)
    nxt = torch.multinomial(probs, num_samples=1) # SAMPLE, do not argmax
    idx = torch.cat([idx, nxt], dim=1)
```

Knob	What it does
temperature $\rightarrow 0$	approaches greedy: repetitive, often degenerate
temperature = 1	samples from the model's own distribution
temperature > 1	flatter, more surprising, less coherent
top_k	restricts to the k most likely tokens before sampling

Sampling is not a detail of the model. Two very different outputs from the same weights differ only in these two numbers, so any claim about “what the model does” has to state them. A surprising fraction of published prompting results are partly results about a temperature setting.

A useful measurement, and one you can do on a laptop: **surprisal**, the negative log probability the model assigned to the token that actually occurred. It is the quantity that correlates with human reading times, which is the whole basis of using language models as models of language processing.

```
logp = torch.log_softmax(model(idx).logits, dim=-1)
surprisal = -logp[0, :-1].gather(1, idx[0, 1:, None]) / math.log(2) # in bits
```

3 Generative models, two objectives

3.1 VAE: the reparameterization trick and the ELBO

The encoder outputs a *distribution* over codes rather than a code. You cannot backpropagate through “draw a sample”, so you rewrite the sample as a deterministic function of the parameters plus fixed noise:

$$z = \mu + \sigma \odot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \mathbf{I}).$$

Now the randomness sits in ε , which needs no gradient, and μ and σ are on the differentiable path.

```
mu, logvar = encoder(x).chunk(2, dim=1) # predict log-variance, not variance
std = torch.exp(0.5 * logvar)
z = mu + std * torch.randn_like(std) # the reparameterization trick
xhat = decoder(z)

recon = F.mse_loss(xhat, x, reduction='sum')
kl = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())
loss = recon + beta * kl
```

The network predicts `logvar` rather than the variance so that its output is unconstrained — a variance must be positive, and `exp` enforces that for free. The two loss terms pull against each other: reconstruction wants codes that are distinct and informative, the KL wants them close to a standard normal so that the space is smooth and samplable. `beta` sets the exchange rate, and the whole character of the model follows from it.

The failure mode with a name. If the KL term dominates — large `beta`, or a decoder powerful enough to reconstruct without help — the encoder collapses to the prior, $\mu \rightarrow 0$ and $\sigma \rightarrow 1$, and the latent code carries *nothing*. Samples still look fine because the decoder learned the data on its own. This is posterior collapse, and the diagnostic is to check whether the KL term goes to zero.

3.2 Diffusion: one closed form and a five-line loss

Adding Gaussian noise repeatedly has a closed form, so you never simulate the forward process step by step:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon.$$

That is the whole reason training is cheap: pick a random t , jump straight there, and ask the network to name the noise.

```
t = torch.randint(0, T, (x0.shape[0],))
eps = torch.randn_like(x0)
xt = abar[t].sqrt()[ :, None] * x0 + (1 - abar[t]).sqrt()[ :, None] * eps
loss = F.mse_loss(model(xt, t), eps) # predict the noise you added
```

The training objective is a mean squared error. That is all it is. The complexity of diffusion models lives entirely in the *sampling* loop, which runs the learned denoiser backwards many times, and in the architecture of the denoiser — not in the loss.

4 Exercises

1. You delete the $\sqrt{d_k}$ from scaled dot-product attention. Training works at $d_k = 8$ and fails at $d_k = 512$. Why?
2. You apply a causal mask by multiplying the attention weights by 0 *after* the softmax. What is wrong?
3. You take softmax over $\text{dim}=-2$ instead of $\text{dim}=-1$. Does anything error? What does the model now compute?
4. A colleague shows an attention map and says it proves the model bases its answer on the word “not”. Give two objections.
5. Generation at $\text{temperature}=0.1$ produces the same sentence repeated forever. Explain in terms of the sampling loop.
6. Your VAE’s KL term drops to almost zero within a few epochs, and reconstructions look good. What has happened, and is the model useful?
7. Diffusion training is a plain MSE. Where did all the complexity go?

Answers

1. Dot products of d_k -dimensional vectors have magnitude growing like $\sqrt{d_k}$. At $d_k = 512$ the unscaled scores are large, the softmax saturates to nearly one-hot, and its gradient is nearly zero, so nothing trains. At $d_k = 8$ the scores are small enough that it hardly matters.
2. The softmax has already distributed weight to the masked positions, so zeroing afterwards leaves rows that do not sum to one — and the unmasked positions get less than their share. Mask with $-\infty$ *before* the softmax.
3. Nothing errors: the matrix is (T, T) , so both axes are valid. You now normalize down the columns, so each row is no longer a distribution over positions and the output is a weighted combination with weights that do not sum to one. It trains, badly, with no diagnostic.
4. (i) There are many heads and many layers; “the attention map” is one choice among dozens that disagree. (ii) Attention weight is not importance — a position contributes its *value* vector, which may be near zero, and the residual stream carries information around the block regardless.
5. At low temperature the softmax is nearly one-hot, so sampling is effectively argmax . The model deterministically picks its most likely continuation, enters a state it has seen before, and cycles. Raising the temperature or using top_k with sampling breaks the loop.
6. Posterior collapse: the encoder has matched the prior and the latent code carries no information, with the decoder reconstructing on its own. It is not useful as a *representation*.

– z tells you nothing about x – even though samples may look fine. Lower beta, or weaken the decoder.

7. Into the sampling loop, which runs the learned denoiser backwards across many steps, and into the denoiser architecture. The training objective is genuinely just “predict the noise you added”, which is why diffusion models are much easier to train than GANs.

Cheat sheet

You want	You write
attention scores	<code>Q @ K.transpose(-2,-1) / sqrt(d_k)</code>
a causal mask	<code>masked_fill(mask == 0, float('-inf'))</code> <i>before softmax</i>
the softmax axis	<code>dim=-1</code> – over keys, always
split into heads	<code>view(N,T,H,d//H).transpose(1,2)</code>
sample a token	<code>torch.multinomial(probs, 1)</code> – not <code>argmax</code>
surprisal in bits	<code>-log_softmax(...).gather(...)</code> / <code>log(2)</code>
the reparameterization	<code>mu + torch.exp(0.5*logvar) * randn_like(mu)</code>
the KL term	<code>-0.5 * sum(1 + logvar - mu**2 - logvar.exp())</code>
a diffusion training step	<code>mse_loss(model(xt, t), eps)</code>